SNOBOL/CREXX

A SNOBOL4 interpreter written in CREXX

Language Reference and User's Guide

Release 1 • Build 1
for CREXX 1.0.0 beta 3 and later

Publication SNC-0001-0
File No. SNOBOL-01

First Edition (October 2026)

This edition applies to SNOBOL/CREXX Release 1 (build 1, 10 October 2026), an interpreter for the SNOBOL4 language written in CREXX Level B, and to all subsequent builds until otherwise indicated in new editions.

SNOBOL/CREXX is a hobbyist project. It is not a product of, and this manual is not a publication of, the authors or owners of any SNOBOL implementation. SNOBOL4, SPITBOL and CSNOBOL4 are names of languages and products, used here only to identify the language that this software implements and the implementations to which it is compared.

CREXX is an open-source REXX implementation and is maintained by its own community. SNOBOL/CREXX needs it to run.

The project repository holds the interpreter source, the examples and the regression tests from which every statement in this manual can be checked. Comments on this publication may be addressed to the project.

Contents

Preface 1

Chapter 1. Introduction 2
- How SNOBOL/CREXX is built 2
- Status 2

Chapter 2. Installing and Running 4
- What you need 4
- Installation steps 4
- Running a program 4
- Command-line options 4
- Input data 4
- Exit status 5
- Testing 5

Chapter 3. The SNOBOL4 Language 6
- Statements 6
- Lines, comments and continuation 6
- Gotos 6
- Names and variables 7
- Data types 7
- Constants 7
- Operators 7
- Unary operators 8
- Keywords 8

Chapter 4. Pattern Matching 9
- Building patterns 9
- Pattern primitives 9
- Assigning matched text 9
- Replacement 10
- Backtracking 10
- Recursive and deferred patterns 10
- Anchored matches 11

Chapter 5. Functions and Data Structures 12
- Defining functions 12
- Arrays 12
- Tables 12
- Programmer-defined data types 13
- Names and indirection 13
- CODE and EVAL 13

Chapter 6. Input and Output 14
- The standard variables 14
- Data after END 14
- Units and files 14
- Output formats 14

Chapter 7. Built-in Functions 15
- Predicates 15
- Strings 15
- Arithmetic 15
- Patterns 15
- Arrays, tables and data 16
- Program and system 16

Chapter 8. Errors 17
- The error limit 17
- Error numbers 17

Chapter 9. How SNOBOL/CREXX Works 18
- Source files 18
- Values 18
- The parser 18
- The matcher 18
- The executor 18
- Adding a function 19

Chapter 10. Restrictions and Known Problems 20

Chapter 11. The C Version **21**
Building and testing 21
Portability rules 21
Numbers and output 21
Differences from the CREXX version 21
Benchmarks 22
Appendix A. Command Summary **23**
Appendix B. Contents of the Repository **24**
Appendix C. Keywords **25**
Appendix D. Sample Programs **26**

Preface

This manual introduces SNOBOL/CREXX, an interpreter for the SNOBOL4 programming language written in CREXX. It tells you how to install and run the interpreter and describes the language it implements: statements and gotos, the data types, the operators, the pattern-matching facility that gives the language its character, programmer-defined functions and data types, and the built-in functions.

It is written for the programmer who knows SNOBOL4 or another string-processing language, and for the newcomer who wants to see what the language can do. Where SNOBOL/CREXX behaves as the classic implementations (SNOBOL4 from Bell Laboratories, SPITBOL and CSNOBOL4) do, this manual says little; where it differs, it says how.

How this manual is organized

- **Chapter 1, Introduction**, says what SNOBOL/CREXX is, how it is built and what it supports.
- **Chapter 2, Installing and Running**, covers the launcher, the command-line options and the regression tests.
- **Chapter 3, The SNOBOL4 Language**, describes statements, names, data types, expressions, operators and keywords.
- **Chapter 4, Pattern Matching**, is the reference for patterns: the primitives, alternation, assignment and backtracking.
- **Chapter 5, Functions and Data Structures**, covers DEFINE, recursion, arrays, tables, DATA types, CODE and EVAL.
- **Chapter 6, Input and Output**, describes the INPUT and OUTPUT variables, units and files.
- **Chapter 7, Built-in Functions**, is the reference for every built-in function.
- **Chapter 8, Errors**, lists the error numbers and explains &ERRLIMIT.
- **Chapter 9, How SNOBOL/CREXX Works**, describes the parser, the matcher and the executor for those who wish to extend the interpreter.
- **Chapter 10, Restrictions and Known Problems**, lists what does not work yet.
- **Chapter 11, The C Version**, describes the portable C89 version of the interpreter, how to build and test it, and how it differs from the CREXX version.
- **Appendix A** summarizes the command line, **Appendix B** lists the files of the repository, **Appendix C** lists the keywords, **Appendix D** lists the sample programs.

Conventions

Program text is shown in monospaced type. In syntax descriptions, uppercase words are written as shown; lowercase words stand for values you supply; items in square brackets [ ] are optional; and an ellipsis ... means that the preceding item may be repeated. SNOBOL4 names are case sensitive unless the -f option is given: TOTAL and Total are different variables.

Examples of programs and terminal sessions are shown in framed figures. In a terminal session the output of the interpreter follows the command that produced it.

Chapter 1. Introduction

SNOBOL/CREXX is a SNOBOL4 interpreter written in CREXX, the open-source REXX implementation. It runs SNOBOL4 programs that use the core language: statements with subject, pattern, replacement and goto fields, the full set of pattern primitives with backtracking, user-defined functions, arrays, tables, programmer-defined data types, indirect reference, and run-time compilation with CODE and EVAL. The interpreter is a single CREXX source file, snobol.crexx, written in CREXX Level B, the statically typed level of the language.

SNOBOL/CREXX gives its users:

- **The pattern language.** Patterns are values. They are built with concatenation and alternation, with ANY, SPAN, BREAK, LEN, ARBNO, BAL and the other primitives, and applied to a string with backtracking until a match is found.
- **Conditional and immediate assignment.** A part of a match is saved with the . operator once the whole match has succeeded, or with $ at the instant it is matched.
- **Recursive patterns.** The unevaluated-expression operator * lets a pattern refer to itself, so that grammars such as nested parentheses can be matched by a single pattern.
- **Functions.** DEFINE creates functions with parameters and locals; they may be recursive. RETURN, FRETURN and NRETURN leave them.
- **Data structures.** Multi-dimensional arrays with chosen bounds, tables indexed by any value, and records declared with DATA.
- **Run-time code.** CODE compiles statements from a string; EVAL evaluates an expression from a string; goto fields may be computed.

How SNOBOL/CREXX is built

A program is read once. Continuation lines are joined, the text is divided into statements, and each statement is parsed into a tree held in parallel arrays: one tree for the subject, one for the pattern, one for the replacement and one for each goto. The parser applies the rules about blanks that are special to SNOBOL4, where a blank is an operator (concatenation) and where a blank is required around a binary operator.

The executor walks the statements. Expressions are evaluated by a recursive evaluator that leaves its result in a set of registers and reports failure through a flag, which is how SNOBOL4 predicates such as LT and IDENT make a statement fail. Patterns are built as trees of nodes by the evaluator and run by a backtracking matcher, which is described in Chapter 9.

The interpreter is compiled by the CREXX compiler to a bytecode file, which the CREXX virtual machine runs. The first run of the launcher performs this compilation; later runs start in under a second.

Status

The figure below shows the major areas of the language and their status in this release.

Area	Content	Status
Statements	Labels, ; separators, continuation lines, comments, success/failure/unconditional gotos, computed gotos	complete
Pattern matching	All primitives, alternation, assignment, cursor, FENCE, ABORT, recursion with *	complete
Data types	String, integer, real, pattern, array, table, name, code, expression, DATA types	complete
Functions	DEFINE, recursion, locals, APPLY, OPSYN	complete
Run-time code	CODE, EVAL, CONVERT to EXPRESSION and CODE	complete
Input and output	INPUT and OUTPUT variables, units, files, ENDFILE, REWIND	complete
Keywords	&ANCHOR, &ERRLIMIT, &STLIMIT and others (Appendix C)	most
Tracing	TRACE, STOPTR, &TRACE	accepted, no effect
External functions	LOAD, UNLOAD	not planned

Area	Content	Status
Operator redefinition	OPSYN of operators	planned

Figure 1. SNOBOL/CREXX language areas

Chapter 2. Installing and Running

What you need

SNOBOL/CREXX needs only the CREXX toolchain: the crexx driver together with the compiler (rxc), assembler (rxas) and virtual machine (rxvm) that it calls. CREXX 1.0.0 beta 3 was used to build and test the interpreter. The driver can be installed from a CREXX release package, or built from source with CMake.

The launcher script snobol is a POSIX shell script. The build script build.py needs Python 3, but only if you change the sources.

Installation steps

1. Install CREXX and make sure the crexx command is on your PATH. If it is not, set the environment variable CREXX to the full path of the driver.
2. Get the SNOBOL/CREXX repository (git clone https://github.com/mikebeer/snobol).
3. Run any program with the launcher: ./snobol examples/hello.sno. The first run compiles the interpreter to bytecode and takes about half a minute. The compiled file, snobol.rxbin, is kept next to snobol.crexx.
4. Run the regression tests with ./tests/run_tests.sh. All of them should report ok.

Running a program

The figure below shows a program run from the launcher.

```
* Count the letters in a string
        S = 'the quick brown fox'
        N = 0
LOOP    S LEN(1) . C =                      :F(DONE)
        IDENT(C,' ')                        :S(LOOP)
        N = N + 1                           :(LOOP)
DONE    OUTPUT = N ' letters'
END

$ ./snobol letters.sno
16 letters
```

Figure 2. A first program, and its output

Without the launcher, give the driver the interpreter source and pass the program after -args:

```
crexx snobol.crexx -args examples/hello.sno
crexx snobol.crexx -nocompile -args examples/hello.sno
```

Figure 3. Running without the launcher

The first form compiles snobol.crexx each time (slow); the second reuses an existing snobol.rxbin.

Command-line options

Options precede the program name. The figure below lists them.

Option	Effect
-f	Fold identifiers to upper case: variable, label and function names are converted, as with the -f option of CSNOBOL4. Without it, names are case sensitive. String values are never changed.
-h	Display a summary of the options and exit.

Figure 4. Command-line options

Input data

Lines that follow the END statement are data. INPUT reads them first, in order, and then continues with the standard input. A program that reads all of its input from the lines after END therefore never waits for the keyboard until it has used them up.

Exit status

The interpreter ends with return code 0 when the program ends normally, and 1 when a runtime error was not absorbed by &ERRLIMIT or when the program could not be loaded or parsed. EXIT(n) ends the program with return code n. An error is reported on the standard error stream in the form:

```
snobol: error 12 at statement 7 (line 9): Division by zero
```

Figure 5. An error message

Testing

The directory tests holds regression programs. Each NAME.sno has an expected-output file NAME.out and optionally NAME.in (standard input) and NAME.flags (interpreter options). tests/run_tests.sh runs them all and prints ok or FAIL for each; with the argument --update it rewrites the expected files, which should be done only after a change has been checked by hand.

Chapter 3. The SNOBOL4 Language

Statements

A program is a sequence of statements, one per line. A statement has up to five fields in a fixed order:

```
label   subject  pattern  = replacement  :goto
```

Figure 6. Statement format

- The **label** starts in the first column. If the line begins with a blank or a tab there is no label. A label is any run of characters up to the first blank.
- The **subject** is the string or variable to be examined or changed.
- The **pattern** is matched against the subject.
- The **replacement** follows an equals sign; the part of the subject that matched is replaced by it. With no pattern, the equals sign makes an assignment.
- The **goto** follows a colon and tells where to go next.

Every field is optional. A statement that has only a subject evaluates it; the statement succeeds unless a predicate in it fails. The figure below shows the forms.

Form	Meaning
subject	Evaluate the expression; used for its effects, such as calling a function.
subject = value	Assign. The statement fails if the value expression fails.
subject pattern	Match the pattern; succeed or fail.
subject pattern = value	Match, and on success replace the matched part by the value.
subject pattern =	Match, and on success delete the matched part.
label ... :goto	Any of the above with a label, a goto, or both.

Figure 7. Statement forms

Lines, comments and continuation

A line that begins with an asterisk is a comment. A line that begins with a minus sign is a control line and is ignored. A line that begins with a plus sign or a period continues the line above it; the plus or period is dropped and the rest is appended. Several statements may be written on one line when separated by semicolons, except inside quoted strings. The first statement of a line with a label carries the label; the others have none.

```
        MSG = 'This is a long '
+               'message'
        A = 1 ; B = 2 ; OUTPUT = A + B
```

Figure 8. Continuation and separators

The statement END, written with the label END or as the line END, marks the end of the program. Control also reaches the end of the program by falling off the last statement or by a goto to the label END.

Gotos

The goto field follows a colon. It holds an unconditional goto (L), a success goto S(L), a failure goto F(L), or both of the last two in either order. A statement succeeds when every part of it succeeded, and fails when a pattern match fails, a predicate fails, or an expression fails.

Goto	Meaning
:(L)	Go to label L unconditionally.
:S(L)	Go to L if the statement succeeded; otherwise fall through.
:F(L)	Go to L if the statement failed; otherwise fall through.
:S(L1)F(L2)	Go to L1 on success, to L2 on failure.

Goto	Meaning
:($X)	Computed goto: the value of X is the name of the label, or a CODE object whose statements are then executed.
:(RETURN)	Leave a function and return its value. FRETURN makes the call fail; NRETURN returns a name.
:(END)	Go to the end of the program.

Figure 9. Goto forms

Names and variables

A variable name begins with a letter and continues with letters, digits, the period and the underscore. A variable needs no declaration; one that has not been assigned has the null string as its value. Variables may hold a value of any type, and the type can change from one assignment to the next.

The variables INPUT and OUTPUT are special: reading INPUT reads a line, and assigning to OUTPUT writes one. TERMINAL writes to the standard error stream. ARB, BAL, REM, FAIL, FENCE, ABORT and SUCCEED start out holding the corresponding patterns.

Data types

DATATYPE	Description
STRING	A string of characters; the null string has length zero.
INTEGER	A whole number held in 64 bits.
REAL	A floating-point number, shown with a decimal point: 2.5, 3., 1.5e-10.
PATTERN	A pattern; see Chapter 4.
ARRAY	A fixed-size array with one or more dimensions.
TABLE	An expandable association table.
NAME	A reference to a variable, array element or keyword, made with the . operator.
CODE	Compiled statements made by CODE.
EXPRESSION	An unevaluated expression made with the * operator.
(type name)	An object of a type defined with DATA.

Figure 10. Data types

Strings convert to numbers when the context asks for a number: the string '12' is the integer 12, '1.5' is a real, and the null string is zero. A number converts to a string when it is concatenated or output. An integer converts to its digits; a real converts with up to 15 digits and always shows a decimal point. A string that is not a number, used in arithmetic, is an error.

Constants

A string constant is enclosed in single or double quotes and cannot contain the quote that delimits it. An integer is a run of digits. A real has a decimal point, an exponent introduced by E, or both: 3.14, 2., .5, 1E3. A minus sign in front of a number constant is part of the constant.

Operators

The figure below lists the operators in order of precedence, the lowest first. A binary operator is written with a blank on each side; a blank between two operands with no operator is concatenation.

Operator	Meaning
? (binary)	Pattern match as an expression; the value is the null string.
|	Alternation (patterns).
blank	Concatenation of strings or patterns.
+ -	Addition, subtraction.
/	Division; integer division when both operands are integers (truncated toward zero).
*	Multiplication.

Operator	Meaning
** or !	Exponentiation; groups from the right.
. $ (binary)	Conditional and immediate assignment of the matched text to a name.
- + ? \ . $ * @ (unary)	Negation, numeric conversion, interrogation, not, name, indirection, unevaluated expression, cursor.

Figure 11. Operator precedence

Integer arithmetic stays integer: 10 / 4 is 2. If either operand is real the result is real. An integer raised to a negative integer power gives 0 except for the bases 1 and -1.

Unary operators

Operator	Meaning
-X	The negative of X.
+X	X converted to a number.
?X	The null string if X succeeds; fails if X fails. It is used to turn a value into a predicate.
\X	Not: the null string if X fails; fails if X succeeds.
.X	The name of X (a NAME value). X may be a variable, an element X<I> or a keyword.
$X	Indirection: the variable whose name is the value of X. It may be assigned to.
*X	The unevaluated expression X, evaluated later (inside a pattern, or by EVAL).
@X	In a pattern: assign the current cursor position to X.

Figure 12. Unary operators

Subscripts are written with angle brackets or square brackets: A<3>, T['name'], M<I,J>.

Keywords

Keywords begin with an ampersand. Some can be assigned. Appendix C lists them. &ANCHOR, when set to 1, makes every pattern match start at the beginning of the subject. &STLIMIT stops the program with an error after that many statements. &ALPHABET is the string of all 256 characters and &LCASE and &UCASE are the lower- and upper-case letters.

Chapter 4. Pattern Matching

A pattern describes a set of strings. The statement S P tries to find, in the string S, a substring that is in the set. The search starts at the first character and moves right until a match is found, unless &ANCHOR is 1. Within one starting point the matcher tries the alternatives in order; if what follows fails it backs up and tries the next alternative. The matched region is described by two cursor positions: a cursor of 0 is before the first character.

Building patterns

A string used where a pattern is expected is a pattern that matches itself. Two patterns written side by side match in sequence. A vertical bar offers alternatives, tried left to right.

```
        S = 'the quick brown fox'
        S ('quick' | 'slow') ' ' ('brown' | 'red') . COLOR
        OUTPUT = COLOR

brown
```

Figure 13. Concatenation and alternation

Pattern primitives

Pattern	Matches
LEN(n)	Exactly n characters.
POS(n)	The null string, if the cursor is n characters from the left end.
RPOS(n)	The null string, if the cursor is n characters from the right end.
TAB(n)	Everything from the cursor up to position n from the left; fails if the cursor is beyond it.
RTAB(n)	Everything from the cursor up to position n from the right.
REM	The remainder of the subject.
ANY(s)	One character that is in the string s.
NOTANY(s)	One character that is not in the string s.
SPAN(s)	The longest run of one or more characters from s. It does not give back characters.
BREAK(s)	The longest run of characters, possibly none, up to but not including a character of s. Fails if there is none.
BREAKX(s)	As BREAK, but on backtracking it extends to the next character of s.
ARB	Any string, shortest first; on backtracking one more character at a time.
ARBNO(p)	Zero or more repetitions of p, fewest first.
BAL	A shortest non-null string with balanced parentheses; on backtracking, the next longer one.
FAIL	Never matches; forces backtracking.
SUCCEED	Always matches, even when backed into.
ABORT	Ends the match at once with failure; no further starting points are tried.
FENCE	Matches the null string; if backed into, the whole match fails (like ABORT).
FENCE(p)	Matches p, but p cannot be backed into once it has matched.

Figure 14. Pattern primitives

Assigning matched text

The binary operator . (conditional assignment) saves the text matched by its left operand in the variable named on its right, but only after the whole pattern has matched. The operator $ (immediate assignment) stores the text as soon as the left operand matches, even if the overall match later fails. The left operand is a pattern; the right operand is a variable, an array or table element, a field, or a keyword.

```
        'abcabc' ('b' | 'c') $ IM 'a'
        OUTPUT = IM

c
```

Figure 15. Conditional and immediate assignment

The unary operator @ assigns the cursor position, an integer, immediately.

```
        'hello world' 'o' @A 'w'
        OUTPUT = A

8
```

Figure 16. The cursor operator

Replacement

When a statement has a replacement, the part of the subject that matched is replaced by the value of the replacement, if the match succeeds. The subject must then be a variable or an element. If the match fails nothing changes. The replacement can be null, which deletes the matched part. The replacement is evaluated only after the match has succeeded.

```
        S = 'the quick brown fox'
        S 'quick' = 'slow'
        OUTPUT = S

the slow brown fox
```

Figure 17. Pattern replacement

Backtracking

The matcher is a backtracking one. When part of a pattern fails, it returns to the most recent choice point and tries the next possibility. The primitives with choice points are the alternation, ARB, ARBNO, BAL, BREAKX and SUCCEED. SPAN, BREAK, ANY, LEN and the others match in one way only. Consider:

```
        'hello world' ARB . A 'o' ARB . B RPOS(0)
        OUTPUT = A '|' B

hell| world
```

Figure 18. Backtracking in ARB

ARB first matches the null string, then one character, and so on, until 'o' follows it. Since the first 'o' is the fifth character, A is hell and B is the rest.

Recursive and deferred patterns

An expression written *X inside a pattern is evaluated when the matcher reaches it, not when the pattern is built. This lets a pattern refer to a variable that has not yet been assigned, including the variable that holds the pattern itself.

```
        P = '(' *P ')' | 'x'
        '((x))' POS(0) P RPOS(0)    :F(NO)
        OUTPUT = 'nested ok'        :(END)
NO      OUTPUT = 'no'
END

nested ok
```

Figure 19. A recursive pattern

A deferred expression can also call a function, for example *PUSH(N), which runs the function at that point in the match. The function's side effects are not undone if the matcher backs up, so patterns that use them

must be written with care: use $ for assignments needed inside the pattern, and do not rely on them when ARBNO or alternation may retry.

Anchored matches

A pattern that starts with POS(0) matches only at the start of the subject. This is how a function that consumes the front of a string is written; without it the pattern would find a match anywhere. The calculator example in Appendix D uses it throughout.

```
        S = '12+3'
        S POS(0) SPAN('0123456789') . N =
        OUTPUT = N ' ' S

12 +3
```

Figure 20. Anchoring with POS(0)

Chapter 5. Functions and Data Structures

Defining functions

DEFINE takes a prototype string and, optionally, an entry label: DEFINE('NAME(P1,P2)L1,L2', 'ENTRY'). The names in parentheses are the parameters; those after the parenthesis are local variables. If no entry label is given, the function begins at the label that has the name of the function. The call evaluates its arguments, saves the parameters, the locals and the function name, assigns the arguments, and starts the statements at the entry point.

A function returns by a goto to RETURN, FRETURN or NRETURN. The value of the variable whose name is the function name is the value returned. FRETURN makes the call fail, so that the statement that used it fails. Because parameters and locals are saved and restored, functions may call themselves.

```
        DEFINE('FACT(N)')                :(MAIN)
FACT    FACT = EQ(N,0) 1                 :S(RETURN)
        FACT = N * FACT(N - 1)           :(RETURN)
MAIN    OUTPUT = FACT(10)
END

3628800
```

Figure 21. A recursive function

Calling an undefined function is an error. DEFINE may be called again with a name that is already defined, which replaces the definition. ARG(name,n) and LOCAL(name,n) return the names of the parameters and locals, APPLY calls a function whose name is a string, and OPSYN gives a function a second name.

Arrays

ARRAY(proto, value) creates an array. The prototype is a number (an array of that many elements indexed from 1), a pair lo:hi, or a comma-separated list of such bounds for several dimensions: '3', '0:9', '2,3', '1:5,0:4'. Every element starts with the given value, or the null string. An element is written A<I> or A<I,J>. An index out of range makes the reference fail; it is not an error.

```
        A = ARRAY('2,2','z')
        A<2,2> = 'q'
        OUTPUT = A<1,1> A<2,2> ' ' PROTOTYPE(A)

zq 2,2
```

Figure 22. Arrays

An array assigned to another variable is shared, not copied; use COPY to copy. SORT(A) and RSORT(A) return a sorted copy of a one-dimensional array.

Tables

TABLE() creates a table. Any value may be a subscript: T<'apple'>, T<3>, T<A>. A string and a number with the same appearance are different keys ('1' and 1). Referring to a key that has never been assigned gives the null string. SORT(T) returns an n-by-2 array of keys and values ordered by key; CONVERT(T,'ARRAY') gives the same pairs in the order in which they were added, and CONVERT(A,'TABLE') turns such a pair array back into a table.

```
        COUNT = TABLE()
        COUNT<'a'> = COUNT<'a'> + 1
        COUNT<'a'> = COUNT<'a'> + 1
        OUTPUT = COUNT<'a'>

2
```

Figure 23. Counting words with a table

Programmer-defined data types

DATA('NODE(VALUE,NEXT)') defines a type with two fields. It creates a constructor function NODE and a field function for each field name. NODE(1,2) creates an object; VALUE(P) gives the field, and VALUE(P) = 3 assigns to it. A field that has not been given is null.

```
        DATA('NODE(VAL,NEXT)')
        L = NODE(1, NODE(2, NODE(3)))
        P = L
LOOP    OUTPUT = VAL(P)
        P = NEXT(P)
        DIFFER(P)                   :S(LOOP)
        END

1
2
3
```

Figure 24. A linked list

NEXT(P) at the end of the list is the null string, which is why the loop stops with DIFFER(P). DATATYPE(P) returns the type name, here NODE, and FIELD('NODE',n) returns the name of field n.

Names and indirection

The unary dot makes a NAME value; $ turns a name or a string back into a variable. With them a program can pass variables by reference and build variable names at run time.

```
        X = 'Y'
        $X = 42
        OUTPUT = Y

42
```

Figure 25. Indirection

CODE and EVAL

EVAL(s) parses the string s as an expression and returns its value. CODE(s) compiles one or more statements, separated by semicolons, and returns a CODE value. A goto to a CODE value, written :($NAME) where NAME holds the code, executes those statements; the program ends when the compiled statements are used up. Both report a syntax error in the string as an error.

```
        OUTPUT = EVAL('1 + 2 * 3')

7
```

Figure 26. EVAL

Chapter 6. Input and Output

Input and output are done by assigning to and reading from variables that have been associated with files.

The standard variables

OUTPUT is associated with the standard output at the start of the program. Each assignment to OUTPUT writes the value, converted to a string, as one line. INPUT is associated with the standard input; each reference to INPUT reads the next line and the reference fails at the end of the input. TERMINAL writes lines to the standard error stream.

```
LOOP    OUTPUT = INPUT      :S(LOOP)
END
```

Figure 27. Echoing the input

Data after END

Lines that follow the END statement are read by INPUT before any standard input. They are not parsed as statements.

Units and files

INPUT(.V, unit, , 'file') associates the variable V with a file opened for reading; OUTPUT(.V, unit, , 'file') opens a file for writing. The unit is a number from 0 to 255. Units 5, 6 and 7 are the standard input, standard output and standard error and need no file name. Referring to V then reads a line, or assigning to V writes one. A call with a unit that is not open and no file name fails.

```
        OUTPUT(.F, 8, , 'out.txt')
        F = 'line one'
        F = 'line two'
        ENDFILE(8)
        INPUT(.G, 9, , 'out.txt')
RD      X = G                   :F(DONE)
        OUTPUT = 'read: ' X     :(RD)
DONE    END

read: line one
read: line two
```

Figure 28. Writing and reading a file

ENDFILE(unit) closes a file that was opened by a call of INPUT or OUTPUT, and REWIND(unit) reopens it from the start. A file that is open for output is reopened for append by REWIND. Lines are text lines ending in a line feed; a carriage return before it is removed on input.

Output formats

An integer is written as its digits. A real is written with up to 15 significant digits and always contains a decimal point, so 3.0 is written 3. and 0.5 as 0.5. A pattern, an array or a table cannot be written as a string; assigning one to OUTPUT writes an empty line.

Chapter 7. Built-in Functions

The figures in this chapter list the built-in functions by purpose. A predicate is a function that returns the null string if it succeeds and fails otherwise, so that it can be tested in the goto field. Arguments that are omitted are the null string. A function name that is also a variable can be used as both.

Predicates

Function	Succeeds when
IDENT(a,b)	a and b are the same type and value.
DIFFER(a,b)	a and b are not identical.
EQ NE LT LE GT GE (a,b)	The numeric comparison holds. Strings are converted to numbers; an unconvertible argument is an error.
LEQ LNE LLT LLE LGT LGE (a,b)	The same, comparing a and b as strings in the character order.
INTEGER(x)	x is an integer, or a string that converts to one.
REAL(x)	x is a real.

Figure 29. Predicates

Strings

Function	Result
SIZE(s)	The number of characters.
SUBSTR(s,i,n)	n characters from position i (the first is 1). Fails if the substring is not within s; with n omitted, to the end.
DUPL(s,n)	s repeated n times.
REVERSE(s)	The characters of s in reverse order.
REPLACE(s,from,to)	s with each character of from replaced by the character at the same place in to. The lengths must be equal.
TRIM(s)	s without trailing blanks.
LPAD(s,n,c) RPAD(s,n,c)	s padded to length n on the left (or right) with the character c, or blanks.
CHAR(n)	The character with code n.
CONVERT(x,type)	x converted to STRING, INTEGER, REAL, NUMERIC, PATTERN, EXPRESSION, NAME, CODE, ARRAY or TABLE; fails if it cannot be.

Figure 30. String functions

Arithmetic

Function	Result
ABS(x)	The absolute value.
REMDR(a,b)	The remainder of a divided by b, with the sign of a; both must be integers.
SQRT(x) EXP(x) LN(x)	Square root, e to the power x and natural logarithm (errors for a negative square root or non-positive logarithm).
SIN(x) COS(x) TAN(x) ATAN(x)	Trigonometric functions of an angle in radians; ATAN gives the arc tangent.
CHOP(x)	A real with its fractional part removed.
RANDOM(n)	A pseudo-random integer from 0 to n-1, or a real from 0 to 1 when n is omitted or zero.

Figure 31. Arithmetic functions

Patterns

The constructors for patterns are those of the figure in Chapter 4: LEN, POS, RPOS, TAB, RTAB, ANY, NOTANY, SPAN, BREAK, BREAKX, ARBNO, FENCE; the constants ARB, BAL, REM, FAIL, ABORT, SUCCEED and FENCE are variables.

Arrays, tables and data

Function	Result
ARRAY(proto,value)	A new array.
TABLE()	A new, empty table.
ITEM(a,i,...)	The element of an array or table. The same as the subscript.
COPY(x)	A copy of an array, table or object; other values are returned unchanged.
SORT(x) RSORT(x)	A sorted copy of an array (or key/value rows for a table), ascending or descending. Numbers sort before strings.
PROTOTYPE(a)	The prototype string of an array, or the item count of a table.
DATA(proto)	Define a data type.
DATATYPE(x)	The type name of x, in upper case.
FIELD(type,n)	The name of field n of a defined type.

Figure 32. Data functions

Program and system

Function	Result
DEFINE(proto,entry)	Define a function.
ARG(f,n) LOCAL(f,n)	The name of the nth parameter or local of f.
APPLY(f,a1,...)	Call the function whose name is the string f.
OPSYN(new,old)	Make new another name for the function old.
EVAL(s)	Evaluate the expression in s, or an unevaluated expression.
CODE(s)	Compile statements in s.
INPUT(.v,unit,len,file) OUTPUT(.v,unit,fmt,file)	Associate a variable with a file or unit.
ENDFILE(u) REWIND(u)	Close or reopen a file.
CLEAR()	Set all variables to the null string.
DUMP()	Write the variables that hold strings or numbers as name = value lines.
EXIT(n)	End the program with return code n.
TIME()	Elapsed milliseconds.
DATE()	A date string (fixed in this release).

Figure 33. Program functions

The functions TRACE, STOPTR, DETACH, BACKSPACE, EJECT and COLLECT are accepted and have no effect. LOAD and UNLOAD give an error.

Chapter 8. Errors

When an error occurs, the interpreter prints a message on the standard error stream, stops the program and ends with return code 1. The message gives the error number, the statement number and the source line:

```
snobol: error 5 at statement 12 (line 14): Undefined function called: FOO
```

Figure 34. An error

The error limit

If the keyword &ERRLIMIT is set to a positive number, an error does not stop the program. The statement in which it occurs fails instead, the number of the error is placed in &ERRTYPE, and &ERRLIMIT is reduced by one. When it reaches zero the next error is fatal. This is how a program can test a conversion that may fail.

```
        &ERRLIMIT = 3
        X = 1 / 0                   :S(BAD)F(OK)
OK      OUTPUT = 'failed ' &ERRTYPE
BAD     END

failed 12
```

Figure 35. Surviving an error

Error numbers

The numbers are those of this interpreter and are not those of any other implementation.

Number	Message
1	Illegal data type (for example a non-numeric string in arithmetic)
2	Output channel not open
3	Erroneous assignment to a keyword
4	Illegal data type for a pattern or for concatenation
5	Undefined function, or a value that is not a variable where one is required
8	Bad argument (bad array prototype, DEFINE, CHAR, DUPL, DATA and so on)
12	Division by zero
13	Illegal arithmetic (square root or logarithm of a bad value)
14	Syntax error in a string given to EVAL or CODE
15	Stack overflow (function calls nested too deeply)
22	Statement limit (&STLIMIT) exceeded
24	Undefined label, or a goto target that failed to evaluate

Figure 36. Error numbers

Syntax errors in the program are found while it is read, before any statement is run, and are reported with the source line; the program does not start.

Chapter 9. How SNOBOL/CREXX Works

This chapter is for the reader who wishes to read or extend the interpreter. The source is in the directory src; build.py joins its files into snobol.crexx.

Source files

File	Contents
a_head.crexx	The main program, the declaration and initialisation of all shared state (procedure init).
b_parse.crexx	Loading the program, statement splitting, label and goto parsing, expression parser.
c_val.crexx	Value registers, conversions, variables, names, arrays, tables, data types, line input and output.
d_pat.crexx	Pattern construction and the backtracking matcher.
e_eval.crexx	Arithmetic, lvalues, expression evaluation, function dispatch, DEFINE, EVAL and CODE.
e_eval2.crexx	The built-in functions.
f_run.crexx	Statement execution, gotos, predefined variables.
consts.txt	Symbolic constants for node types, pattern node types and keyword codes.

Figure 37. Source files

build.py assembles the files in order, builds the list of shared variables from the declarations in init, numbers the built-in functions from the word list in init, replaces the symbolic constants with numbers, and writes snobol.crexx.

Values

Every value is held in four registers: a type code, a string, an integer and a real. The integer register also carries the identity of objects such as patterns, arrays and tables. Variables, array elements and the arguments of a function call are stored in parallel arrays of the same four parts. A failing expression sets a flag, which every caller tests.

The parser

SNOBOL4 gives meaning to blanks, so the parser works on the text of a statement, not on a stream of tokens. An operator between two operands must have a blank on both sides, or on neither. A blank not next to such an operator separates two operands and is the concatenation operator. The expression parser is a precedence-climbing loop that takes these rules into account.

The matcher

A pattern is a tree of nodes in parallel arrays. Matching is done by one procedure, match_at, with three pieces of state: the node being matched, the cursor, and a continuation, a linked list of nodes still to be matched. Concatenation pushes the right-hand node on the continuation and goes to the left-hand node. Alternation saves a choice point holding the second alternative and goes to the first. When a primitive fails the matcher pops the most recent choice point and resumes there. ARB, ARBNO, BAL and BREAKX use special kinds of choice point that re-enter them with a longer match.

Conditional assignments are put in a list and carried out only when the whole match has succeeded; the list is cut back when the matcher backs up past them. An immediate assignment is done at once. Because the matcher keeps its state on explicit stacks rather than on the CREXX call stack, long subjects and deep backtracking do not overflow the stack, and a deferred pattern may itself run a match.

The executor

Statements are numbered in order. exec_from runs from a statement, evaluating the parts and then following the goto. A call of a DEFINEd function saves the variables of the function on a save stack, assigns the arguments and calls exec_from recursively at the entry label; the RETURN goto leaves that call.

Adding a function

1. Add the name to the bfn word list in a_head.crexx. The position in the list is the function's number, and build.py makes the symbolic constant FN_NAME for it.
2. Add a branch for FN_NAME in the select statement of the procedure builtin in e_eval2.crexx. The arguments are in the argument stack at base+1 to base+na; the result is left in the registers; to fail, set fail to 1.
3. Run python3 build.py, then ./tests/run_tests.sh.

Chapter 10. Restrictions and Known Problems

SNOBOL/CREXX implements the core of SNOBOL4. The following are known limits.

- **Operators cannot be redefined.** OPSYN works for function names only, and binary & is not an operator.
- **Tracing** (TRACE, STOPTR, &TRACE, &FTRACE) is accepted but does nothing.
- **Structured output** (the format argument of OUTPUT) and record lengths in INPUT are ignored. Files are text files; there is no binary or random access.
- **External functions** (LOAD, UNLOAD) are not supported.
- **Real numbers** have the precision of the CREXX floating-point type, about 15 digits, and are shown in that form; the last digits of results such as 2 ** 0.5 may differ from other implementations.
- **Integers** are 64-bit (the width of a CREXX integer). Overflow is not reported as an error: the CREXX virtual machine drops the result of an overflowing addition, subtraction or multiplication and the interpreter then uses the previous result of the same operation (0 at first), so 9223372036854775807 + 1 gives 0 on a fresh run. The C version imitates this exactly.
- **Two CREXX defects.** In snobol.crexx, ITEM(A, i) on an array and EVAL of an unevaluated expression value (*X, or CONVERT(s, 'EXPRESSION')) with sub-expressions misbehave, because a global register is passed as a parameter and changes under the callee. A literal of 19 or more digits, a real string such as 1e999 and a huge real passed to LEN or SUBSTR stop the program with a CREXX PANIC. The C version evaluates these correctly (a literal that is too large becomes a real; the others are errors).
- **CODE objects** run to the end of the program: when the compiled statements are used up, the program ends.
- **Side effects in patterns.** A function called from a pattern with *F() is not undone when the matcher backs up, as in other implementations.
- **Speed.** The interpreter is a program running on a virtual machine. Typical statements take a few microseconds; heavy pattern matching on very long strings is slower than in compiled SNOBOL4.
- **DATE** returns a fixed string.
- **Case.** Without -f, names are case sensitive, and the keywords and function names must be written in upper case.

Chapter 11. The C Version

The directory c of the repository holds a second implementation of the same interpreter, written in strict C89. It was written so that SNOBOL4 can run on VM/370 (CMS), where only a small C compiler is available, and it follows the layout and rules of the other C ports in the author's repositories (apl, Smalltalk, Python). It is a direct port of the CREXX sources: the same parser, the same backtracking matcher, the same evaluator, the same error numbers and messages. It passes the regression tests of this repository with output identical to that of the CREXX version, and starts in a few milliseconds.

Building and testing

```
cd c
make                # builds ./snobol from the single file snobol_all.c
make test           # runs ../tests against ./snobol
./snobol [-f] prog.sno

make strict         # gcc and clang strict flags, sanitizers, 32-bit long
make deck           # 71-column card images in deck/snobol_deck.c
make deck-test      # runs the tests against the deck build
make bench          # runs ../bench with the C (and the CREXX) version
```

Figure 38. Building the C version

The sources are in c/src, in seven parts (base and strings, values, parser, patterns, evaluator, built-in functions, main). tools/amalg.sh inlines them into one self-contained file, snobol_all.c, which is checked in so that the program builds with a single compiler command. tools/gen_consts.py regenerates the constant header from src/consts.txt and the function list of the CREXX sources, so that both versions share their numbering.

Portability rules

- **Strict C89.** No // comments, no declarations after statements, no long long, no stdint.h and no C99 library functions. The program compiles without warnings under gcc and clang with -std=c89 -pedantic-errors -Wall -Wextra -Wconversion -Wshadow.
- **One external symbol.** All functions are static except main, so that the eight-character external names of CMS are no problem.
- **EBCDIC safe.** Character classes are literal character sets; nothing relies on the ASCII code points. Only the digits are tested with '0' to '9'.
- **32-bit long.** On VM/370 a long may have 32 bits. Integers are long, overflow is detected explicitly, and the test suite passes in a 32-bit build (gcc -m32). Integer overflow then happens at 2147483647.
- **No operating system services** except, with -DSNO_POSIX, gettimeofday for TIME().
- **Card images.** tools/deck.py folds the amalgamated file to 71 columns without changing the token stream, and checks the result, for the VM/370 build.
- **Checked** with AddressSanitizer and UBSan on all tests and on a differential comparison against the CREXX version: about forty hand-written programs and 1500 random expressions.

Numbers and output

Real numbers print exactly as the CREXX version prints them: at most 15 significant digits, trailing zeros removed, plain notation for exponents from -6 up to 17 and d.ddddE+XX beyond, then the SNOBOL conventions of a decimal point and an upper case E (3., 0.3, 1.E+20, 1.5E-10, inf.). The algorithm is the one of the CREXX run-time library. Integer overflow, which the CREXX virtual machine does not report, is imitated as described in Chapter 10, so that programs that overflow print the same numbers with both versions.

Differences from the CREXX version

- **Bytes, not characters.** CREXX strings are UTF-8 and SIZE and SUBSTR count code points; the C version counts bytes. &ALPHABET is the 256 raw bytes. ASCII text behaves identically.

- **Defects that are not copied.** ITEM on arrays, EVAL of an unevaluated expression, and the numeric conversion panics described in Chapter 10 work in the C version.
- **TIME()** returns elapsed milliseconds with SNO_POSIX and processor time otherwise.
- **Floating point.** The last digit of results of SIN, EXP, ** and the like depends on the host's mathematical library; on IEEE hosts it agrees with CREXX. A VM/370 build uses hexadecimal floating point, where the 15 printed digits can differ in the last place.

Benchmarks

The directory bench holds four deterministic programs that both versions run with identical output: loops (integer and real arithmetic, Collatz, gcd), strings (concatenation, SPAN, BREAK, ARBNO, BAL and replacement), tables (inserts, lookups, SORT, arrays) and recursion (Fibonacci, Ackermann, Towers of Hanoi, permutations through DEFINE). sh bench/run.sh runs them and compares the output with the stored results; the C version needs between 0.02 and 0.12 seconds for each, the CREXX version between 2 and 7.

Appendix A. Command Summary

```
./snobol [-f] program.sno
crexx snobol.crexx -nocompile -args [-f] program.sno

-f   fold names to upper case
-h   show usage
```

Figure 39. Command line

The environment variable CREXX names the crexx driver used by the launcher.

Appendix B. Contents of the Repository

Path	Contents
snobol	Launcher script.
snobol.crexx	The interpreter, generated by build.py (generated; checked in so that the repository runs at once).
build.py	Assembles snobol.crexx from src.
src/	The interpreter sources and consts.txt.
examples/	Sample programs.
tests/	Regression tests and run_tests.sh.
docs/	This manual as a Word document and as a PDF.
c/	The portable C89 version: sources in c/src, the amalgamated snobol_all.c, the card deck, Makefile, tools and tests/run_tests.sh.
bench/	Benchmark programs (loops, strings, tables, recursion) with their expected output and run.sh.
README.md	Short description of the project.

Figure 40. Repository

Appendix C. Keywords

Keyword	Meaning	Assignable
&ANCHOR	1 = anchored matching, 0 = scan (default)	yes
&TRIM	Trailing blank handling (kept, no effect)	yes
&FULLSCAN	Kept, no effect	yes
&ERRLIMIT	Number of errors that are survivable	yes
&ERRTYPE	Number of the last error	yes
&STCOUNT	Number of statements run	yes
&STLIMIT	Statement limit; -1 = none	yes
&STNO	Number of the current statement	no
&LASTNO	Number of the statement that did the last goto	no
&FNCLEVEL	Depth of nested function calls	no
&ALPHABET	The 256 characters	no
&LCASE &UCASE	The lower- and upper-case letters	no
&MAXLNGTH	Maximum string length (kept, not enforced)	yes
&DIGITS	15	no

Figure 41. Keywords

Appendix D. Sample Programs

The directory examples holds the programs in the figure below. Each one is also a regression test.

File	Shows
hello.sno	OUTPUT
wordfreq.sno	TABLE, SPAN and BREAK patterns, REPLACE, SORT, data after END
sieve.sno	ARRAY, loops with gotos, IDENT
calc.sno	Recursive functions, anchored patterns, a recursive-descent parser
roman.sno	DEFINE, BREAK with replacement, loops

Figure 42. Sample programs

Besides these, tests/patterns.sno, tests/functions.sno, tests/features.sno, tests/errors.sno and tests/fold.sno exercise the pattern primitives, functions, data structures, error handling and the -f option.